

COMP3173 25F

Project Description

Table of Contents

Prologue	1
Language Feature	2
Examples	5
Phase 1 – Lexical Analysis	7
Phase 2 – Syntax Analysis	7
Phase 3 – Semantic Analysis	8
Input / Output	9
Difficulty Level	14
Bonus	15

Prologue

Project Title: Math is Fun

Grouping:

This is an **individual** project.

Objective:

In this project, you will develop a small analyzer for some math expressions. This language is simplified, which only consists of numbers, Booleans, number operators, relational operators, variable declarations, function declarations, and function applications. This language is designed to be simple enough, making it easy for students to implement. Before commencing work on the project, it is imperative that you thoroughly review this **long** document.

The first part of this document explains the language's features with examples, which is like a programming language user manual. By reading it, students will gain a comprehensive understanding of the language, enable them to write expressions in this language, and to identify errors in source codes. The subsequent part of the document outlines the instructions of implementation by different phases. Students are expected to complete the project by sequentially completing 3 phases.

Implementation Language:

C, Java, or Python, you have your own choice. Startup codes and instructions for both

languages are given in the package. The implementation can only use standard library. Any other external libraries are not included in the grading environment.

Grading:

The grades are distributed to each phase and depends on the difficulty level. The metric is as follows.

Difficulty	Submission	Compilation	Lexing	Parsing	Typing	AST	Total
Level 1	5%	5%	25%	18%	4%	4%	61%
Level 2	5%	5%	30%	22%	6%	6%	74%
Level 3	5%	5%	35%	26%	8%	8%	87%
Level 4	5%	5%	40%	30%	10%	10%	100%

It is not a shame if you cannot finish all parts of this project. By estimation, 50% (Level 1 Lexing + Parsing) is sufficient to let a student pass (without a downgrade in the exam). And 80% (Level 4 Lexing + Parsing) is enough to get an A- (with a similar grade in the exam).

Bonus:

- type check recursive functions – 1%
- type check partial function calls – 1%

Cheating:

Fail the course. **One-line copy is copy.**

DDL:

We do not put DDLs for each phase. The DDL for the entire project will be **Dec. 19 midnight**, the last day of classes. But, **do not wait until the last minute**. Please expect that the project will take **20 - 30 student hours**. Goliath spent 5 hours to implement the sample solution.

Students will submit their works to **AUTOLAB** at http://172.31.13.233/auth/users/sign_in. Autolab can only be visited via BNBK local network. If you want to visit the website, BNBK VPN is required. Marking will be purely based on testcases.

The submission link will be ready soon.

Testcases:

Sample answers are already given in the package with three different versions for Windows, Linux, and MacOS. **Please try it!** Students can also write their own testcases to understand the language better.

Language Feature

A complete piece of source code in this language has

- zero, one, or multiple **declarations** and
- exactly one **calculation statement**.

A declaration can be declaring either a **variable** or a **function**.

A **variable declaration** is in the form

Let T id be E .

where

- “**Let**” is a keyword to initiate a declaration;
- “**T**” is a basic type (will be explained later);
- “**id**” gives the name of a variable, which is a sequence of English letters in the lower case except keywords;
- “**be**” is a keyword which acts as the assignment operator in other programming languages.
- “**E**” is constant in type **T**.
- “**.**” is a full stop to finish the sentence.

For example,

Let bool x be true .

declares a variable **x** in type **bool** of value **true**.

A **function declaration** is

Let F id maps I to E .

where most elements are same a variable declaration except

- “**F**” is a **type expression** for constructed function type;
- “**maps**” is similar to “**be**”, but for function declaration;
- “**I**” is a list of variable names;
- “**to**” connects function arguments with function expressions, for function declaration;
- “**E**” is a calculation expression.

A **function type expression** is

G >> T

where

- “**G**” is a list of basic types for the domain of the function;
- “**>>**” is a connector to and
- “**T**” is a basic type (same as variable declaration) for the codomain of the function.

A **list** (of types or variable names) consists of one or multiple elements. Two elements are separated by a comma “**,**”.

For example, a full function declaration can be

Let num , num >> num plus maps x , y to x + y .

where

- “**plus**” is the function name;
- “**num, num >> num**” is the type expression for function “**f**”;
- “**num, num**” is the domain (also understood as the Cartesian product **num × num**);
- “**num**” is the codomain;
- “**x , y**” is the argument list; and
- “**x + y**” is a calculation expression (will be explained later).

A calculation statement is

Compute E .

where

- “**Compute**” acts as the main function in other programming languages, which initialize a calculation; and
- “**E**” is a calculation expression.

A calculation expression can be one of

- “**id**”: an atomic expression, the name of an identifier;
- “**lit**”: an atomic expression, a constant literal;
- “**exp op exp**”: a compound expression, where “**exp**” is an arbitrary calculation expression, and “**op**” is a binary operator;
- “**(exp)**”: a compound expression, for declaring sub-structures;
- “**if exp1 then exp2 else exp3**”: a compound expression, for flow control;
 - “**exp1**” is the guard and of type bool;
 - “**exp2**” and “**exp3**” are calculation expression of type number;
- “**exp1 exp2**”: function calls – calling the function “**exp1**” on the input “**exp2**”.

Binary operators are

- “**+**” number addition;
- “*****” number multiplication;
- “**<**” relational operator, returns “**true**” if the left operand is smaller than the right operand and “**false**” otherwise; and
- “**=**” relational operator, returns “**true**” if two operands are the same and “**false**” otherwise. This operator is polymorphic. The two operands can be both numbers or Booleans.

Operator precedence in this language is same as arithmetic.

A complex example of computation statement can be

Compute if (plus 1 2) < 1 then 0 else 1 + 2 * 3 .

where

- “**plus 1 2**” calls function “**plus**” (from the previous example) on input “**1**” and “**2**”;
- “**if ... then ... else ...**” is the if statement; and
- “**1 + 2 * 3**” means “**1 + (2 * 3)**”.

“if” statements can be nested, but need parentheses to specify the structure. For example,

```
if true then (if true then 1 else 2) else 3
```

Types:

- **num:**
 - Basic type
 - A single-digit integer is an arbitrary decimal number.
 - A multi-digit integer starts with a non-zero decimal number and followed by any sequence of arbitrary decimal numbers.
 - Negative numbers have the negative sign “-” in the front. Positive numbers do not have.
- **bool:**
 - Basic type
 - This type only two constants: “true” and “false”
 - A Boolean constant can be specified by users or by the evaluation on a relational operator. For example,

```
3 < 5
```

is of type **bool** because “<” is a relational operator. The behavior of “<” will be explained later in this document.
- **NA:**
 - Basic type
 - **Cannot** be declared by users
 - For expressions without any type
For example, a declaration sentence does not have a specific type. So, it is of type **NA**.
- function type:
 - Constructed type, for functions
 - The function type declared by users are in the shape “... , ... >> ...”, but the compiler (what students are implementing) should present the type in the form “... -> ... -> ...”. In this way, the type of a function and its argument list can be unified in the same form.
The symbol “->” is not included in our language because it is already captured by grammar production rules.
 - For example, if we have

```
Let num , num >> num times maps x , y to x * y .
```

the compiler will present the type of “times” as “num->num->num”.

Examples

- Example 1:

```
Compute 0 .
```

One of the simplest examples.

- Example 2:

```
Let num x be -3 .  
Compute x + 1 .
```

Example 2 declares a number “**x**”, whose value is “**-3**”. And it does a simple calculation.

- Example 3

```
Let num x be 3 .  
Let bool y be false .  
Let num >> num f maps a to a + x .  
Compute if y then x else f x .
```

This example is about variable declarations, function declarations, if statements, and function calls.

- Example 4

```
This is a lexical error .
```

Example 4 has a lexical error because the letter “**T**” is not in the alphabet. “**This**” is not a keyword, so it has to be an identifier. However, identifiers are strings of English letters in lower case. Capital letters are excluded.

- Example 5

```
this is a syntax error .
```

In this example, every word will be tokenized as an identifier. So, there is no lexical error. But, its structure is incorrect.

- Example 6

```
Let bool x be 4 .  
Compute x .
```

This is an example of type error. “**x**” is declared of type “**bool**”, but “**4**” is of type “**num**”. Their types are not matched.

- Example 7

```
Let num a1 be 1 .  
Compute a1 .
```

This is a syntax error because “**a1**” is recognized as identifier “**a**” and number literal “**1**”. But in a declaration “**be**” should right after the identifier.

Some other examples are given in the package for each difficult level. More examples should be written by students. The sample answer is already given in the package. Students can check the analyzing outcome by themselves.

Phase 1 – Lexical Analysis

Timeline: Students can start working on Phase 1 after “Lecture 3 – Finite Automata”.

Description:

In the first phase, students need to implement a lexer for this project. The lexer reads the source code as a stream of characters, cuts them into lexemes, classifies lexemes as tokens, and decides attributes for some lexemes. The lexer also reports lexical error (spelling mistakes) if the source code contains any. Tokens are previously defined in the language user manual and summarized again as below. The tokens that contain exactly one lexeme each are

- Keywords: `Let`, `be`, `maps`, `to`, `if`, `then`, `else`, `Compute`, `num`, `bool`
- Punctuations: `[`, `]`, `.`, `(`, `)`
- Arithmetic operators: `+`, `*`
- Relational operators: `=`, `<`
- Type constructor: `>>`

And the tokens that contain infinite lexemes are

- `num_lit`: a signed number literal
 - A single-digit integer is an arbitrary decimal number.
 - A multi-digit integer starts with a non-zero decimal number and followed by any sequence of arbitrary decimal numbers.
 - If the number starts with `-`, it is negative; otherwise, it is positive.
 - ~~○ Users can only declare non-negative integers. Negative integers are constructed through subtraction operations.~~
- `bool_lit`: Boolean constant, either `true` or `false`
- `identifier`: user defined names, variables or functions

Space “ ”, indentation “\t”, and newline “\n” are special characters, which are ignored by the lexer but they can terminate other lexemes by force. To simplify this project, you can assume two lexemes are **ALWAYS separated by at least one space**.

Symbol table also needs to be implemented in this phase. Symbol table is for recording **identifiers only**. Each identifier needs to be with few attributes for Phase 3 – Semantic Analysis. So, it is possible that students cannot decide which attribute should be recorded currently. This is very common. You can modify the symbol table later.

We do not grade symbol table. So, how to implement this feature is totally up to students.

Phase 2 – Syntax Analysis

Timeline: Students can start working on Phase 2 after “Lecture 8 – SLR Parser”

Description:

This is the second phase of a compiler. Students are expected to implement an **SLR parser** for this language. The language grammar and the corresponding SLR Parsing table are already given in the package. You don’t need to do the conversion.

The parsing table is stored as an “CSV” file. Thus, we highly recommend that your program

should read the parsing table as an external text document. If you hardcode the parsing table into the parser, that will be very difficult to modify it later. Especially, we have setup 4 difficulty levels for this project. Different difficulty levels result different languages. Some students may want to upgrade or downgrade difficulty. More about the levels will be later.

The parser works **with a stack**, which contains the already **parsed configurations**. The parser also needs to maintain a **syntax tree** to present the grammatical structure of the source code. A substructure of the tree is created when the parser reduces its stack. Each node in the syntax tree has some attributes, which are left uninitialized in Phase 2, but will be used in Phase 3 Semantic Analysis.

Phase 3 – Semantic Analysis

Timeline: Students can start working on Phase 3 after “Lecture 12 – Type System Part 2”. In fact, Lecture 11 and 12 are examples of semantic analysis for type checking, which is rather easy. Students are encouraged to self-study them and start working on Phase 3 as early as possible.

Description:

Semantic analysis is the last phase for the front-end of a compiler, which is the last piece of this project. In this part, the semantic analyzer **checks the types** of the source code and **generates a syntax tree**. Type errors are reported if there is any.

Some semantics (**type checking** rules) of the language may be inherited, but it is L-attributed. Thus, we recommend that the semantic analyzer should be implemented as a function, which

- contains the semantic definitions for all syntactic rules, and
- execute one specific semantic calculation each time when the parser reduces its stack using the corresponding grammar rule.

The result of type checking is stored by the attributes of each structure in the stack and the parse tree of the parser. (Recall that each structure of a grammar is combined with two attributes: type and value in Phase 2.) Thus, your parser needs to be modified a little to interact with your semantic analyzer.

Again, function types are declared as “`... , ... >> ...`”, but recognized as “`... -> ... -> ...`” by the analyzer. For example,

```
Let num, num >> num f maps x, y to x + y .
```

declares a function “`f`” of type “`num -> num -> num`”.

Expressions with **undeclared variables** are recognized as **type error** because undeclared variables are of type unknown.

Some type checking rules are already given in the package. Students can use them directly. Rest of the rules are left to students themselves.

Syntax tree generation is similar to the procedure given in Lec 10 Page 18 – 22. The syntax tree is obtained from the parse tree after type checking by following rules.

1. All the non-terminals (the internal nodes) are removed.
2. All punctuations (like full stop, and parentheses) are removed.
3. Expressions with operators (“`+`”, “`*`”, “`=`”, “`<`”, “`>>`”, “`,`”) use the operator as the

parent. For example, the syntax tree of “1 + 2” has “+” as the root with the first child “1” and the second child “2”.

4. “>>” and “,” are regarded as operators (also called connectives). “>>” connects two type expressions. “,” connects two type expressions or two argument lists.
5. “If” statements use “if” as the parent with the guard as the first child, the expression on “if” branch as the second child, and the expression on “else” branch as the third child. No need to present “else” and “then” in the syntax tree.
6. For functions and function applications, an artificial node “Fun” is created as the parent.
7. “Let” and “Compute” are the roots for declaration and calculation respectively.
8. In fact, the outcome may not be a single syntax tree, but a forest. One tree presents a single declaration or calculation statement.

Input / Output

Your program should run with a command line argument, which specifies the input source code file name. **All source code files DO NO have extension.** For example in Windows, if the compiled execution file is called “fun.exe” and the source code is “in1”, then your program can be executed by “fun.exe in1”. Input files can be edited by any text editor.

The output of your compiler consists of the followings.

1. **No requirement on screen output.** This is where you can make fun. Enjoy!
2. **A file named “xxx_lexer.json”** by file write, where “xxx” is the input source code name, for example “in1_lexer.json”.

This file contains the outcome of the lexer (**lexemes** and the corresponding **tokens**) as **a list of dictionaries** in the syntax of JSON. A list of objects is enclosed by a pair of square brackets “[]”. And a dictionary consists of two pairs and enclosed by a pair of curly brackets “{ }”. The first pair is a string “name” and the token name. The second pair is a string “lexeme” and the lexeme. For example,

```
Compute 3 .
```

the output file should be

```
[
  {
    "name": "Compute",
    "lexeme": "Compute"
  },
  {
    "name": "num_lit",
    "lexeme": "3"
  },
  {
    "name": ".",
    "lexeme": "."
  }
]
```

Note that spaces, indentations, and line breaks are not required. They are only used to improve the readability. Thus, the above example can also be

```
[{"name":"Compute","lexeme":"Compute"},
 {"name":"num_lit","lexeme":"3"},{"name":".","lexeme":"."}]
```

3. A file named “**xxx_parser.json**” “**xxx_parse.json**” by file write

The outcome of Phase 2 must be written into “**xxx_parser.json**” “**xxx_parse.json**” using parentheses to present the hierarchical structure of the parse tree. Each non-terminal in the parse tree is a dictionary of two pairs. The first pair presents the name of the non-terminal. The second pair presents a list of children. Each terminal is presented in the same way as the outcome of the lexer. For example,

```
Compute 3.
```

the output file should be

```

{
  "name": "Z",
  "children": [
    {
      "name": "L",
      "children": [
        {
          "name": "C",
          "children": [
            {
              "name": "Compute",
              "lexeme": "Compute"
            },
            {
              "name": "E",
              "children": [
                {
                  "name": "A",
                  "children": [
                    {
                      "name": "M",
                      "children": [
                        {
                          "name": "U",
                          "children": [
                            {
                              "name": "P",
                              "children": [
                                {
                                  "name": "num_lit",
                                  "lexeme": "3"
                                }
                              ]
                            }
                          ]
                        }
                      ]
                    }
                  ]
                }
              ]
            }
          ]
        },
        {
          "name": ".",
          "lexeme": "."
        }
      ]
    }
  ]
}

```

Again, spaces, indentations, and line breaks are for readability only.

4. A file named "`xxx_type.json`" by file write
This file contains the type-checked parse tree in the same structure as "`xxx_parse.json`". But each node in the tree contains one more attribute named "`type`". For terminals, the order of attributes is "`name`", "`type`", and "`lexeme`". For non-terminals, the order is "`name`", "`type`", and "`children`". For the above example, the output file will be

```

{
  "name": "Z",
  "type": "NA",
  "children": [
    {
      "name": "L",
      "type": "NA",
      "children": [
        {
          "name": "C",
          "type": "NA",
          "children": [
            {
              "name": "Compute",
              "type": "NA",
              "lexeme": "Compute"
            },
            {
              "name": "E",
              "type": "num",
              "children": [
                {
                  "name": "A",
                  "type": "num",
                  "children": [
                    {
                      "name": "M",
                      "type": "num",
                      "children": [
                        {
                          "name": "U",
                          "type": "num",
                          "children": [
                            {
                              "name": "P",
                              "type": "num",
                              "children": [
                                {
                                  "name": "num_lit",
                                  "type": "num",
                                  "lexeme": "3"
                                }
                              ]
                            }
                          ]
                        }
                      ]
                    }
                  ]
                }
              ]
            }
          ]
        }
      ]
    },
    {
      "name": ".",

```

```

        "type": "NA",
        "lexeme": "."
      }
    ]
  }
]
}

```

5. A file named “`xxx_ast.json`” by file write

Similar to “`xxx_type.json`”, this file holds the syntax tree constructed from the parse tree. The AST for the above example is

```

[
  {
    "name": "Compute",
    "children": [
      {
        "name": "2"
      }
    ]
  }
]

```

To visualize JSON files, students can use this website <https://jsontree.vercel.app/>

Error Handling

This project does not correct any error. The entire program stops at the first that it can find and put an error message in output JSON files. The error messages are “`Lexical error`”, “`Syntax error`”, and “`Type error`” for the three corresponding errors. An error only affects the current phase and the following phases. Previous phases should produce results normally. For example, if the input source file has a syntax error but no lexical error; “`xxx_lexer.json`” contains all the lexemes and tokens, but “`xxx_parser.json`” “`xxx_parse.json`”, “`xxx_type.json`”, and “`xxx_ast.json`” should only contain “`Syntax error`”.

Difficulty Level

This project has 4 difficulty levels, for students with different aims. Each level corresponds to a subset of the language. Once level 4 is finished, the entire project is accomplished. Phase 2 – syntax analysis heavily depends on the difficulty level. Different levels have different grammars and different parsing tables. If a student hardcodes the parsing table into the parser and wants to upgrade the level, the parser has to be implemented again. So, you need to make your decision wisely.

- Level 1 is for the students who want to pass this course. This level is only about calculations without if statements. The included features are

- keyword “**Compute**”,
 - number literals and Boolean literals,
 - binary operators “**+**”, “*****”,
 - punctuations “**{**”, “**}**”, “**,**”, and
 - corresponding syntax and semantics.
- Level 2 is for the students who want to learn this course, but don’t have enough time for this project. This level includes
 - all features from level 1,
 - “**if...then...else...**” statements, and
 - relational operators “**=**”, “**<**”.
- Level 3 should be most of the selections. This level adds variable declarations to level 2. Thus, the new features are
 - keywords “**Let**”, “**num**”, “**bool**”, “**be**”,
 - identifiers and symbol table (Yes! If you only want to finish level 1 or 2, symbol table is not even required), and
 - corresponding syntax and semantics.
- Level 4 is the complete project, which includes function declarations and function calls. The new features are
 - keywords “**maps**”, “**to**”,
 - type / argument constructors “**>>**”, “**,**”, and
 - corresponding syntax and semantics.
 - But, all functions in level 4 are non-recursive and single-argument. Recursion and multi-argument functions are bonus.

Sample Answer

The sample answer with three versions is already included in the package. Put the version for your operating system in the same folder as the input source file and command line execute the program with the input file name as the argument. Then, the output JSON files will be created in the same folder. For example, if the OS is Windows and the input file is “**in1**”, the command will be “**sample_windows.exe in1**”.

Bonus

$$\lambda y.(\lambda x.y(xx))(\lambda x.y(xx))$$

“In order to understand recursion, one must understand recursion.”

The bonus of this project consists of two features.

Bonus 1 - Recursive functions

The syntax of this language allows the following declaration

```
Let num >> num ft maps x to if x < 10 then x * ft ( x + 1 ) else
10 .
```

which is a **recursive function** “ft”. Recursive functions make this language general purposed. To type check the recursive function in this example, the analyzer should understand the type of “ft” as “num -> num” after the parser reduces $F \rightarrow G \gg T$. Then, when the analyzer checks the type of “ft” in “x * ft (x + 1)”, its type can be found in the symbol table.

Bonus 2 - Partial function call

The syntax for function calls is $U \rightarrow U P$. And P can directly derive some literals or identifier by $P \rightarrow \text{num_lit} \mid \text{bool_lit} \mid \text{identifier}$. Thus, arguments are not included by a pair of parentheses. For example, let “f” be a two-variable function of type “num -> num -> num” and “x”, “y” are two arguments in the correct types. Calling this function in C language is “f(x,y)”; but such function call becomes “f x y” in our language.

Furthermore, the syntax $U \rightarrow U P$ is left recursive, meaning that “f x y” is understood as “(f x) y”. The function call is **left associative**. Then, the type checker verifies the type of “f x” before the entire expression. What is the type of “f x”? The following example shows the answer.

First, let’s declare a two-variable function which is named “add” and adds two number.

```
Let num, num >> num add maps x, y to x + y .
```

Then, declare a single-variable function which is named “addthree” and adds three to the input. But, this function is implemented by a function call to “add”.

```
Let num >> num addthree maps x to add x 3 .
```

The behavior of “addthree x” is exactly same as “add 3 x” for any number “x”. Thus, “addthree” and “add 3” are of the same type “num -> num”. And “add 3” **partially calls** function “add” on “3”.

To type check partial function calls, the type rule for $U1 \rightarrow U2 P$ can be

```
if U2.type == A -> B and P.type == A then
  U1.type := B
else
  U1.type := type_error
```

Note that B can be a function type, not only a basic type.

Bonus Grading

- 1% for recursive functions
- 1% for partial function calls